

Lab 08 – DataGrid, RadioButton, CheckBox, Vertical Menu

Title: Advanced Data Display and Filtering Controls

Objectives

This lab focused on displaying employee data in a structured table format using the DataGrid control. I also added filtering capabilities using RadioButton and CheckBox controls, and introduced a vertical navigation menu. The goal was to create a more functional interface that allows users to view and filter data efficiently.

Implementation

MainWindow.xaml

```
<Window x:Class="SalariiUI.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:model="clr-namespace:SalariiModel;assembly=SalariiModel"
    Title="Sistem Salarii - Gestionare Angajați"
    Height="700"
    Width="1100"
    WindowStartupLocation="CenterScreen">

    <Window.Resources>
        <SolidColorBrush x:Key="PrimaryBrush" Color="#2C3E50"/>
        <SolidColorBrush x:Key="AccentBrush" Color="#3498DB"/>
        <SolidColorBrush x:Key="DangerBrush" Color="#E74C3C"/>
        <SolidColorBrush x:Key="LightBrush" Color="#ECF0F1"/>
    </Window.Resources>
</Window>
```

```
<Style x:Key="MenuButton" TargetType="Button">
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="HorizontalAlignment" Value="Stretch"/>
    <Setter Property="Padding" Value="15,10"/>
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Cursor" Value="Hand"/>
    <Style.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
            <Setter Property="Background" Value="#34495E"/>
        </Trigger>
    </Style.Triggers>
</Style>
```

```
<Style TargetType="DataGrid">
    <Setter Property="FontSize" Value="13"/>
    <Setter Property="AlternatingRowBackground" Value="#F5F5F5"/>
    <Setter Property="RowHeight" Value="30"/>
    <Setter Property="HeadersVisibility" Value="Column"/>
    <Setter Property="CanUserAddRows" Value="False"/>
    <Setter Property="CanUserDeleteRows" Value="False"/>
    <Setter Property="AutoGenerateColumns" Value="False"/>
</Style>
```

```
</Window.Resources>
```

```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="180"/>
```

```

        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <!-- Vertical Menu -->

    <StackPanel Grid.Column="0" Background="{StaticResource PrimaryBrush}">

        <Label Content="💰 Salarii" FontSize="22" FontWeight="Bold"
            Foreground="White" HorizontalAlignment="Center" Margin="10,20" />

        <Button Content="🏠 Acasă" Style="{StaticResource MenuButton}" />
        <Button Content="✚ Adaugă" Style="{StaticResource MenuButton}" />
        <Button Content="📋 Listă" Style="{StaticResource MenuButton}" />
        <Button Content="🔍 Caută" Style="{StaticResource MenuButton}" />
        <Button Content="📁 Rapoarte" Style="{StaticResource MenuButton}" />
        <Separator Background="#34495E" Margin="10,10" />
        <Button Content="⚙ Setări" Style="{StaticResource MenuButton}" />
        <Button Content="✖ Ieșire" Style="{StaticResource MenuButton}"
Background="{StaticResource DangerBrush}" />
    </StackPanel>

    <!-- Main Content -->

    <Grid Grid.Column="1" Margin="15">

        <Grid.RowDefinitions>

            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <!-- Filter Controls -->

        <Border Grid.Row="0" Background="{StaticResource LightBrush}" Padding="10"
Margin="0,0,0,10" CornerRadius="5">

```

```

<StackPanel>

    <TextBlock Text="Filtrare angajați" FontWeight="Bold" FontSize="14"
Margin="0,0,0,10"/>

    <WrapPanel>

        <GroupBox Header="Tip Salariu" Margin="0,0,15,0">

            <StackPanel>

                <RadioButton x:Name="rbToate" Content="Toate" IsChecked="True"
GroupName="TipFiltru" Margin="5" Checked="RadioButton_Checked"/>

                <RadioButton x:Name="rbLunar" Content="Lunar" GroupName="TipFiltru"
Margin="5" Checked="RadioButton_Checked"/>

                <RadioButton x:Name="rbOrar" Content="Orar" GroupName="TipFiltru"
Margin="5" Checked="RadioButton_Checked"/>

            </StackPanel>

        </GroupBox>

        <CheckBox x:Name="chkBonus" Content="Doar cu bonusuri" Margin="15,5"
Checked="CheckBox_Changed" Unchecked="CheckBox_Changed"/>

        <Button Content="Aplică filtre" x:Name="btnFiltreaza" Click="BtnFiltreaza_Click"

            Background="{StaticResource AccentBrush}" Foreground="White"

            Padding="15,5" Margin="10,0"/>

    </WrapPanel>

</StackPanel>

</Border>

<!-- DataGrid -->

<DataGrid Grid.Row="1" x:Name="dgAngajati" ItemsSource="{Binding Angajati}"

    SelectedItem="{Binding AngajatSelectat, Mode=TwoWay}" Margin="0,10">

    <DataGrid.Columns>

        <DataGridTextColumn Header="ID" Binding="{Binding Id}" Width="50"/>

        <DataGridTextColumn Header="Nume" Binding="{Binding Nume}" Width="100"/>

        <DataGridTextColumn Header="Prenume" Binding="{Binding Prenume}"
Width="100"/>

        <DataGridTextColumn Header="Funcție" Binding="{Binding Funcție}" Width="120"/>

```

```

        <DataGridTextColumn Header="Departament" Binding="{Binding Departament}"
Width="100"/>

        <DataGridTextColumn Header="Tip" Binding="{Binding TipSalariu}" Width="80"/>

        <DataGridTextColumn Header="Brut" Binding="{Binding SalariuBrutEfectiv,
StringFormat={}{0:C}}" Width="100"/>

        <DataGridTextColumn Header="Net" Binding="{Binding SalariuNet,
StringFormat={}{0:C}}" Width="100"/>

        <DataGridTextColumn Header="Data Angajării" Binding="{Binding DataAngajarii,
StringFormat=dd.MM.yyyy}" Width="120"/>

    </DataGrid.Columns>

</DataGrid>

<!-- Status Bar -->

<StatusBar Grid.Row="2" Margin="0,10,0,0">

    <StatusBarItem>

        <TextBlock Text="{Binding ElementName=dgAngajati, Path=Items.Count,
StringFormat='Total angajați: {0}'}"/>

    </StatusBarItem>

    <Separator/>

    <StatusBarItem>

        <TextBlock Text="Selectați un angajat pentru detalii..."/>

    </StatusBarItem>

</StatusBar>

</Grid>

</Grid>

</Window>

```

Code-Behind (MainWindow.xaml.cs)

```

using System;

using System.Collections.Generic;

using System.Linq;

```

```

using System.Windows;

using System.Windows.Controls;

using SalariiModel;

using SalariiStocare;


namespace SalariiUI
{
    public partial class MainWindow : Window
    {
        private IStocareData _admin;

        private List<Angajat> _angajatiOriginali;


        public MainWindow()
        {
            InitializeComponent();

            _admin = StocareFactory.GetAdministratorStocare();

            IncarcaAngajati();
        }


        private void IncarcaAngajati()
        {
            _angajatiOriginali = _admin.GetAngajati();

            dgAngajati.ItemsSource = _angajatiOriginali;
        }


        private void RadioButton_Checked(object sender, RoutedEventArgs e)
        {
            // The filter will be applied when the button is clicked
        }
    }
}

```

```
private void CheckBox_Changed(object sender, RoutedEventArgs e)
{
    // The filter will be applied when the button is clicked
}
```

```
private void BtnFiltreaza_Click(object sender, RoutedEventArgs e)
{
    AplicaFiltre();
}
```

```
private void AplicaFiltre()
{
    var rezultat = _angajatiOriginali.AsEnumerable();

    // Filter by salary type (RadioButton)
    if (rbLunar.IsChecked == true)
    {
        rezultat = rezultat.Where(a => a.TipSalariu == TipSalariu.Lunar);
    }
    else if (rbOrar.IsChecked == true)
    {
        rezultat = rezultat.Where(a => a.TipSalariu == TipSalariu.Orar);
    }

    // If "Toate" is selected, no filter is applied

    // Filter by bonus (CheckBox)
    if (chkBonus.IsChecked == true)
    {
        rezultat = rezultat.Where(a => a.Bonusuri > 0);
    }
}
```

```
        dgAngajati.ItemsSource = rezultat.ToList();  
    }  
}  
}
```

Why I Used This Approach

DataGrid: The DataGrid is the standard WPF control for displaying tabular data. It automatically handles sorting, selection, and scrolling. I set `AutoGenerateColumns="False"` to have full control over which properties are displayed and how they're formatted.

Vertical Menu: A StackPanel with styled buttons creates a professional-looking navigation menu. The menu structure sets up the application for future features.

RadioButton Grouping: Using the `GroupName` property ensures only one RadioButton in the group can be selected at a time. This is perfect for mutually exclusive filter options.

CheckBox for Bonus Filter: A CheckBox provides a simple toggle for filtering employees with bonuses.

WrapPanel for Filter Controls: WrapPanel automatically wraps controls when space is limited, making the layout responsive.

Two-Step Filtering: The filter is applied when the user clicks the "Aplică filtre" button, not automatically. This prevents performance issues with large datasets and gives the user control over when filters are applied.

LINQ for Filtering: Where with lambda expressions provides a clean way to build the filter logic step by step.

Currency Formatting: Using `StringFormat="{0:C}"` automatically formats numbers as currency, using the current culture settings.

Testing & Results

I tested the filters by:

1. Adding employees with different salary types and bonus amounts
2. Selecting "Lunar" – only salaried employees were displayed
3. Selecting "Orar" – only hourly employees were displayed
4. Checking "Doar cu bonusuri" – only employees with `bonus > 0` were displayed

5. Combining filters (e.g., "Lunar" + "Doar cu bonusuri") – only salaried employees with bonuses were displayed

The DataGrid correctly displayed all employee data with proper formatting.

Reflection

The DataGrid is a very powerful control that would have taken many lines of code to implement manually. The filtering logic using LINQ is clean and easy to modify. I also appreciated how WPF handles data binding – I just set the ItemsSource and the DataGrid updates automatically. The vertical menu adds a professional feel to the application and sets up a navigation structure for future features. The ability to combine filters using LINQ's chaining capabilities is very powerful and maintainable.